

Let's now go line by line through the notebook **2\_letter\_recognition.ipynb**, which is focused on **recognizing letters using a neural network classifier on the Letter Recognition dataset**.

---

### ☑ Code Cell 1: Import Libraries

```
python
CopyEdit
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical
```

- **pandas**: For data loading and manipulation.
  - **numpy**: For numeric operations.
  - **matplotlib.pyplot**: For plotting data and metrics.
  - **train\_test\_split**: To split dataset into train and test sets.
  - **LabelEncoder**: Converts letter labels to integers.
  - **StandardScaler**: Normalizes features (mean=0, std=1).
  - **Sequential**: Linear stack of layers (for the model).
  - **Dense**: Fully connected neural network layer.
  - **to\_categorical**: One-hot encoding for output labels.
- 

### ☑ Code Cell 2: Load and Inspect Dataset

```
python
CopyEdit
df = pd.read_csv('/content/letter-recognition.data', header=None)
df.head()
```

- Loads the **Letter Recognition dataset** from a CSV file.
  - **header=None**: Dataset doesn't contain a header row.
  - Displays the first 5 rows.
- 

### ☑ Code Cell 3: Data Preparation

```
python
CopyEdit
X = df.iloc[:, 1:].values
y = df.iloc[:, 0].values
le = LabelEncoder()
y = le.fit_transform(y)
```

```
y = to_categorical(y)
```

- `x`: All columns except the first — input features (16 attributes).
  - `y`: First column — target letters (A–Z).
  - `LabelEncoder`: Converts letters (A–Z) to integers (0–25).
  - `to_categorical(y)`: Converts integer labels to one-hot encoding.
- 

## ☑ Code Cell 4: Train-Test Split

```
python
CopyEdit
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

- Splits the data:
    - 80% training.
    - 20% testing.
  - `random_state=42`: Ensures reproducibility.
- 

## ☑ Code Cell 5: Feature Scaling

```
python
CopyEdit
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

- Standardizes features to have zero mean and unit variance.
  - Prevents some features from dominating due to scale differences.
- 

## ☑ Code Cell 6: Build Neural Network Model

```
python
CopyEdit
model = Sequential()
model.add(Dense(64, input_shape=(16,), activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(26, activation='softmax'))
```

- `First Dense(64, ...)`: Input layer with 64 neurons, ReLU activation.
  - `Second Dense(32, ...)`: Hidden layer with 32 neurons.
  - `Output Dense(26, softmax)`: 26 classes (A–Z), softmax for probabilities.
-

## ☑ Code Cell 7: Compile Model

```
python
CopyEdit
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

- `adam`: Optimizer for training.
  - `categorical_crossentropy`: Suitable loss for multi-class classification.
  - Monitors `accuracy` during training.
- 

## ☑ Code Cell 8: Train the Model

```
python
CopyEdit
history = model.fit(X_train, y_train, epochs=30, batch_size=64,
validation_data=(X_test, y_test))
```

- Trains the model:
    - For 30 epochs.
    - With batch size of 64.
    - Validates on test set.
- 

## ☑ Code Cell 9: Evaluate Accuracy

```
python
CopyEdit
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

- Evaluates model on the test set.
  - Prints test accuracy.
- 

## ☑ Code Cell 10: Plot Accuracy

```
python
CopyEdit
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

- Plots training and validation accuracy for each epoch.

---

## ☑ Code Cell 11: Plot Loss

```
python
CopyEdit
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

- Plots training and validation loss over epochs.

---

## 📌 Summary:

- The notebook performs **letter classification (A–Z)** using a simple **multi-layer neural network**.
- Involves preprocessing (encoding and scaling), training, evaluation, and plotting.